

Câu 1: Tại sao trước khi bắt đầu một dự án, việc chọn kiến trúc lại là bước sống còn?

Việc lựa chọn kiến trúc phần mềm (Software Architecture) ngay từ đầu dự án được coi là bước đi sống còn vì các lý do cốt lõi sau:

Định hình bộ khung và hướng đi: Kiến trúc là sơ đồ thiết kế tổng thể. Nếu không có khung vững chắc, việc phát triển code sau này sẽ trở nên vô định, chắp vá và dễ dẫn đến sụp đổ hệ thống khi dự án phình to.

Ảnh hưởng trực tiếp đến chi phí và thời gian: Thay đổi cấu trúc hệ thống ở giai đoạn sau (khi đã viết xong nhiều code) tốn kém gấp hàng chục lần so với việc thiết kế đúng ngay từ đầu.

Đảm bảo các thuộc tính chất lượng phi chức năng: Kiến trúc quyết định hệ thống có thể mở rộng (Scalability) khi có nhiều người dùng hay không, có bảo mật (Security) tốt không, và có dễ bảo trì (Maintainability) không. Code tốt đến mấy mà kiến trúc sai thì hệ thống vẫn không thể vận hành ổn định.

Quản lý rủi ro và giao tiếp đội ngũ: Kiến trúc cung cấp một góc nhìn chung cho tất cả các bên (Lập trình viên, Quản trị dự án, Khách hàng), giúp phát hiện sớm các rủi ro về mặt công nghệ trước khi quá muộn.

Câu 2: Liệt kê các loại kiến trúc phần mềm phổ biến (ít nhất 3 loại)

Dưới đây là 4 loại kiến trúc phần mềm phổ biến nhất hiện nay:

Kiến trúc nguyên khối (Monolithic Architecture): Toàn bộ các chức năng, thành phần của ứng dụng (giao diện, logic, cơ sở dữ liệu) được gộp chung vào một mã nguồn duy nhất. Dễ phát triển lúc đầu nhưng khó mở rộng về sau.

Kiến trúc phân tầng (Layered / N-Tier Architecture): Hệ thống được chia thành các tầng có nhiệm vụ riêng biệt xếp chồng lên nhau (ví dụ mô hình 3 tầng: Presentation Layer - Business Logic Layer - Data Access Layer). Tầng trên chỉ gọi xuống tầng ngay dưới nó.

Kiến trúc vi dịch vụ (Microservices Architecture): Hệ thống được chia nhỏ thành nhiều dịch vụ độc lập, mỗi dịch vụ chạy một chức năng riêng biệt, có cơ sở dữ liệu riêng và giao tiếp với nhau qua API hoặc Message Broker.

Kiến trúc hướng sự kiện (Event-Driven Architecture): Hệ thống vận hành dựa trên việc sản sinh, phát hiện và tiêu thụ các sự kiện (Events). Các thành phần liên kết cực kỳ lỏng lẻo với nhau qua một hệ thống trung gian (như Kafka, RabbitMQ).

Câu 3: Liệt kê và giải thích các thuộc tính chất lượng (Quality Attributes) của phần mềm

Thuộc tính chất lượng (phi chức năng) quyết định hệ thống vận hành tốt đến mức nào dưới các điều kiện thực tế:

Tính mở rộng (Scalability): Khả năng hệ thống chịu tải và xử lý mượt mà khi lượng người dùng hoặc dữ liệu tăng lên đột biến mà không cần phải viết lại code (ví dụ: nâng cấp phần cứng hoặc thêm máy chủ).

Tính sẵn sàng (Availability): Tỷ lệ thời gian hệ thống hoạt động bình thường để phục vụ người dùng (thường đo bằng tỷ lệ % như 99.99%). Hệ thống có khả năng tự phục hồi hoặc dự phòng khi có một máy chủ bị lỗi.

Tính bảo trì (Maintainability): Mức độ dễ dàng để sửa lỗi, cập nhật tính năng mới hoặc tối ưu hóa mã nguồn mà không làm ảnh hưởng đến các phần đang chạy ổn định của hệ thống.

Tính bảo mật (Security): Khả năng chống lại các cuộc tấn công mạng độc hại, bảo vệ toàn vẹn dữ liệu và đảm bảo chỉ những người dùng được cấp quyền mới có thể truy cập thông tin.

Hiệu năng (Performance): Tốc độ phản hồi của hệ thống đối với yêu cầu của người dùng (Response time) và mức độ tiêu hao tài nguyên phần cứng (CPU, RAM) khi xử lý tác vụ.

Đối với một startup, sự thành công của dự án không chỉ nằm ở việc ra mắt sản phẩm thật nhanh (Time-to-market), mà còn ở khả năng sống sót và tăng trưởng dài hạn. "Kiến trúc tốt" chính là nền móng vững chắc quyết định sự thành công đó thông qua mối quan hệ biện chứng sau:

- **Giai đoạn đầu (Sự sống sót):** Một kiến trúc phù hợp (ví dụ Monolith hoặc Layered) giúp startup tối ưu hóa chi phí, tập trung nguồn lực để hoàn thiện tính năng cốt lõi và đưa sản phẩm ra thị trường nhanh nhất nhằm kiểm chứng ý tưởng kinh doanh.
- **Giai đoạn sau (Sự bứt phá):** Khi sản phẩm được thị trường đón nhận, lượng người dùng tăng trưởng nóng, một "kiến trúc tốt" có tính linh hoạt cao sẽ giúp hệ thống dễ dàng mở rộng, chuyển đổi (ví dụ nâng cấp lên Microservices) mà không phải đập đi xây lại từ đầu.

Tóm lại: Kiến trúc phần mềm không phải là một chi phí lãng phí, mà là một khoản đầu tư chiến lược. Chọn sai kiến trúc sẽ tạo ra "nợ kỹ thuật" khổng lồ, khiến dự án sụp đổ khi đang trên đà tăng trưởng. Chọn đúng kiến trúc giúp startup cân bằng giữa tốc độ và sự bền vững, từ đó biến ý tưởng công nghệ thành một sản phẩm kinh doanh thành công.